

Maritime cargospace - Sprint 1

Andrea Passini, Paolo Savino, Alessandra Tolomelli

Luglio 2026

1 Introduzione

Lo Sprint 1 parte da due elementi chiave: l'[architettura logica](#) e la formalizzazione dei [requisiti](#) svolti nel precedente Sprint 0.

Come già anticipato, il goal di questo sprint è quello di implementare un primo prototipo dell'architettura, al termine del quale il sistema dovrà essere in grado di:

- Visualizzare una prima griglia modificabile astratta per la hold e verificarne lo stato delle celle.
- Dettagliare la comunicazione tra cargoservice e IOPort, gestendo gli stati di engaged e disengaged del sistema.
- Primo prototipo mock dei devices.

2 Analisi dei Requisiti

L'obiettivo di questo sprint è lo sviluppo e la verifica del flusso di gestione delle richieste di carico, che costituisce la funzionalità centrale del sistema. Tutte le attività di analisi del dominio e di raccolta dei requisiti necessarie a questo scopo sono già state completate durante lo Sprint 0 e non verranno quindi approfondite ulteriormente.

Poiché alcuni elementi dell'architettura, come il **sonar**, il **led** del **PicoW**, il **cargorobot** e il **markerDevice**, non sono ancora stati implementati, il loro comportamento sarà riprodotto tramite componenti simulati. Questi mock permetteranno di eseguire test e validare il funzionamento del sistema anche in assenza dell'hardware definitivo.

La realizzazione dei componenti reali è prevista negli sprint successivi. Parallelamente, i mock saranno progressivamente aggiornati e resi più aderenti al comportamento effettivo dei dispositivi, prendendo come riferimento la specifica QAK definita nello Sprint 0 e reperibile al seguente [link](#)

3 Analisi del Problema

Prima di procedere con la progettazione dettagliata del software, è fondamentale analizzare i vincoli logici dei componenti, le relazioni di dipendenza e le modalità ottimali per ridurre il divario di astrazione (*abstraction gap*) tra il mondo fisico e il modello computazionale, esplicitando i motivi delle scelte tecnologiche e dei costrutti esclusi.

3.1 Scelta del Robot e Comparazione DDR

L'analisi dei supporti tecnologici forniti per la gestione del Differential Drive Robot (DDR) deve basarsi sulla capacità del componente di rappresentare la realtà operativa della stiva. Le tre opzioni alternative sono state escluse per evidenti limiti architetturali:

- **RobotObj26** è stato scartato in quanto si tratta di un semplice POJO troppo passivo e vincolato strettamente a Java, che costringerebbe ad adottare lo stesso linguaggio per l'intero sistema chiamante.
- **VirtualRobot26** rappresenta unicamente l'ambiente grafico e fisico di simulazione ma manca di un'interfaccia orientata ai messaggi. Interagire con esso avrebbe significato costringere l'applicazione a farsi carico dell'intera cinematica del robot.
- **RobotService26**, pur essendo parzialmente reattivo, avrebbe richiesto che l'applicazione chiamante gestisse manualmente il routing sulla mappa e l'evitamento degli ostacoli passo dopo passo, lasciando un divario di astrazione elevatissimo nel codice di business.

La scelta si orienta necessariamente su **RobotSmart26**, comandato tramite messaggi dispatch di movimento, poiché include nativamente una rappresentazione formale del mondo che mappa fedelmente l'ambiente della stiva, conoscendo a priori la dislocazione geometrica degli slot e degli ostacoli fisici. Al robot non interessa la semantica logica dello slot (se sia libero o prenotato), ma lo tratta come coordinata di destinazione o come ostacolo da evitare. Delegando il calcolo del percorso via algoritmo A^* a questo supporto, si diminuisce l'intervallo di astrazione del software, beneficiando inoltre del blocco immediato del piano di movimento in caso di anomalie sollevate dal sonar.

3.2 Modellazione dei Dispositivi Periferici: Il Sonar e il Led Logico

L'integrazione dei dispositivi periferici richiede una netta separazione tra il livello hardware e la logica applicativa

3.2.1 Analisi della comunicazione con il sonar

Il sonar produce con continuità un dato grezzo (la distanza rilevata), mentre il cargoservice necessita di un'informazione a livello semantico, ovvero la presenza del container o un'eventuale condizione di guasto, senza dover conoscere i dettagli di soglia e unità di misura con cui tale informazione viene derivata. Si pone quindi il problema di stabilire sia il meccanismo di comunicazione tra i due attori, sia la responsabilità della traduzione dal dato meccanicistico all'evento applicativo.

Un primo approccio, basato su **polling** da parte del cargoservice, è stato scartato perché rende l'orchestratore responsabile sia del monitoraggio continuo, attività non pertinente al suo ruolo, sia dell'interpretazione della soglia di rilevamento, violando la separazione delle responsabilità tra i due attori.

Sono state quindi valutate tre alternative circa il grado di consapevolezza reciproca dei due attori:

1. il sonar ignora completamente il contesto applicativo ed emette un dato indifferenziato;
2. il sonar è consapevole del cargoservice e gli invia direttamente un messaggio mirato;
3. il sonar non conosce il **cargoservice**, ma è quest'ultimo ad essere **interessato** alle informazioni del sonar.

Si è scelta la terza alternativa, che corrisponde a un pattern *Observer*: il sensore fisico non possiede alcuna consapevolezza dell'applicazione o del cargoservice, mentre è l'orchestratore a dipendere dai suoi dati, configurandosi come osservatore del sensore. Il sonar rimane così disaccoppiato dalla logica applicativa, e questo disaccoppiamento è reso possibile, a livello infrastrutturale, da un canale *publish/subscribe* su MQTT.

Questa scelta comporta inoltre che la traduzione dal dato grezzo all'informazione semantica avvenga presso il sonar stesso, e non presso il cargoservice. Il sonar deve veicolare tre informazioni chiave: la rilevazione pura della distanza, la presenza di un'entità a una determinata soglia e l'indicazione di un guasto qualora la distanza rimanga superiore al limite di spazio libero ($D > D_{FREE}$) per più di 3 secondi. Per isolare questa logica, si introduce concettualmente un **Sonar Logico**, un QActor intermediario che modella il comportamento del sensore fisico all'interno del sistema applicativo, distinto dal **sonar fisico**, responsabile della sola misura. In questo modo il cargoservice non riceve mai un flusso continuo di dati, ma reagisce esclusivamente a eventi semanticamente rilevanti: presenza del container, condizione di guasto, oppure assenza di variazioni significative.

3.2.2 Evoluzione delle comunicazioni del sonar

In una prima fase di prototipazione, l'interazione può avvenire tramite **Polling** sincrono (Request-Reply) oppure tramite **Dispatch**. Entrambi questi approcci risultano però inefficienti in un sistema distribuito, poiché causano uno spreco di banda dovuto all'invio continuo di messaggi punto-a-punto anche quando la distanza non varia in modo significativo. La strategia ottimale a regime prevede l'impiego di un modello basato su **eventi a sottoscrizione**. A differenza dell'evento QAK classico emesso in broadcast globale a tutto il contesto, questa variante introduce un **pattern publish-subscribe** guidato. Il cargoservice (per la logica di business) e il monitor esterno (per il rilevamento dei guasti) effettuano una sottoscrizione esplicita al flusso informativo del sensore. Il Sonar si limita a pubblicare i dati sul proprio canale senza conoscere l'identità, il numero o la natura dei suoi sottoscrittori. Questa scelta azzerà l'accoppiamento architetturale in perfetto accordo con il pattern **Observer** e ottimizza l'efficacia del sistema evitando la proliferazione di eventi globali sulla rete.

3.2.3 Analisi della comunicazione con il Led

Analogamente al sensore di distanza, anche l'indicatore visivo necessita di un'astrazione software. Il **Led Logico** riceve i comandi asincroni di attivazione o lampeggio dall'orchestratore e si fa carico di pilotare il componente hardware reale o simulato.

3.3 Natura del componente Hold

La modellazione della **hold** non può essere ridotta a un semplice oggetto software tradizionale (POJO) interno al thread del servizio di coordinamento. In uno scenario reale, la stiva subisce richieste asincrone di verifica, prenotazione e rilascio degli slot. Poiché la concorrenza non è garantita nei POJO ordinari (l'accesso simultaneo allo stato degli slot causerebbe race conditions e corruzioni della memoria), è indispensabile che la hold sia modellata come un **Attore QAK**. Questo incapsula lo stato e garantisce che ogni richiesta venga processata in modo sequenziale, atomico e sicuro attraverso la propria coda interna di messaggi.

3.4 Controllo degli accessi all'IOPort

Nello stato `handle_load_request`, per valutare l'accesso al sistema quando arriva una richiesta di carico, la scelta ottimale prevede l'utilizzo delle **guardie native del linguaggio QAk** sulla transizione di ingresso (es. `whenRequest load_request [!ioPortOccupied] -> ...`) anziché l'uso di un costrutto condizionale esplicito in stile Kotlin (`Goto ... if ...`).

- **Gestione dichiarativa e pulizia dello stato:** Se la porta è occupata (`ioPortOccupied == true`), la guardia fallisce e il framework QAk impedisce l'attivazione della transizione verso lo stato di elaborazione. Il **cargoservice** evita così di transitare inutilmente in stati di gestione del rifiuto, mantenendo il codice della macchina a stati estremamente pulito e focalizzato sul solo "flusso felice".
- **Preservazione del messaggio senza risposte forzate:** Al contrario di un costrutto `if` esplicito, che consumerebbe e rimuoverebbe la richiesta dalla coda costringendo il sistema a inviare un rifiuto immediato per liberare il chiamante, la guardia non scarta il messaggio. Se la condizione è falsa, la richiesta rimane congelata o in attesa nella coda dei messaggi dell'attore. Non appena la risorsa si libera e la guardia torna vera, il messaggio viene processato naturalmente, automatizzando la coda d'attesa.

3.5 Disaccoppiamento e organizzazione del sistema

L'analisi architetturale evidenzia la necessità di separare nettamente le responsabilità del sistema in sotto-contesti indipendenti:

- La **Web GUI** viene tenuta strutturalmente fuori dal contesto dei QActor, evitando di inquinare la logica core con le tecnologie di frontend che non appartengono alla rete degli attori.
- I componenti hardware simulati (pico/sensori) e l'interfaccia di input IOPort rappresentano entità isolate e distinte. Questa separazione logica consente una scomposizione del lavoro parallela all'interno del team di sviluppo: ciascun sottosistema viene sviluppato in totale autonomia, per poi confluire armonicamente nell'infrastruttura di comunicazione distribuita basata sul broker MQTT.

4 Piano di Lavoro e Distribuzione dei Compiti

#	Task	Assegnatario	Ore Stimate
1	Analisi del Problema	Savino	4
2	Definizione TestPlan	Savino	3
3	Modellazione mockdevices	Passini	4
4	Orchestratore cargoservice sulla hold	Tolomelli	7
5	Deployment e debug	Passini	8
6	Goal Sprint 2	Tolomelli	1
7	Revisione	Tutti	5

5 TestPlan

I test sono raggiungibili al seguente [link](#).

```
-----  
TEST 1 - Invio della richiesta di carico (simula pressione bottone)  
-----
```

```
State simula_pressione_bottone {  
    println("ioport | [AUTO-TEST] Invio load_request a cargoservice...")  
    color magenta  
    [# displayMessage = "Loading request..." #]  
    request cargoservice -m load_request : loadRequest(none)  
}
```

Questo è il punto di partenza del ciclo di test: IOPort simula la pressione del pulsante da parte di un utente inviando una load-request a cargoservice. Aggiorna, inoltre, il messaggio di display locale per dare un feedback visivo coerente con l'azione appena eseguita. Da qui parte la Transition t1 che smista la risposta in tre possibili rami: `accepted`, `retrylater`, e `refused`.

```
-----  
TEST 2 - Gestione della risposta "load accepted"  
-----
```

```
State accepted {  
    onMsg( load_accepted : loadAccepted(SLOTID) ) {  
        println("ioport | DISPLAY: load accepted - place container in IOPort")  
        color green  
        [# displayMessage = "ACCEPTED - Place container" #]  
    }  
    println("ioport | [AUTO-TEST] Container posizionato fisicamente.  
    Attendo 3 secondi prima del check del sensore...") color magenta  
    delay 3000  
}
```

Verifica il ramo "positivo" del flusso: quando cargoservice risponde con `load_accepted`, IOPort aggiorna il display invitando a posizionare il container. Subito dopo simula il tempo fisico necessario all'operatore per depositare il container con un delay di 3 secondi, prima di passare al test successivo che simula la rilevazione del sensore.

```
-----  
TEST 3 - Simulazione rilevamento sensore (container detected)  
-----
```

```
State accepted {  
    onMsg( load_accepted : loadAccepted(SLOTID) ) {  
        println("ioport | DISPLAY: load accepted - Il sistema verificherà la presenza  
        del container tramite il sonar") color green  
        [# displayMessage = "ACCEPTED - Placing container..." #]  
    }  
}  
Goto transfer_in_progress
```

Simula il posizionamento fisico del container sulla pedana dell'IOPort dopo che la richiesta di carico è stata accettata. Questo test serve a verificare la capacità del cargoservice di rilevare autonomamente l'oggetto attraverso l'interrogazione attiva del sonar, dimostrando che il sistema sa riconoscere la presenza del carico ed avviare il robot senza bisogno di stimoli esterni.

TEST 4 - Gestione della risposta "retry later"

```
State retrylater {
    onMsg( load_retrylater : loadRetryLater(none) ) {
        println("ioport | DISPLAY: retry later") color yellow
        [# displayMessage = "RETRY LATER" #]
    }
    println("ioport | [AUTO-TEST] Hold momentaneamente occupata,
    attendo 3s e riprovo...") color magenta
    delay 3000
}
```

Testa il ramo in cui la hold risulta temporaneamente occupata: il display mostra "RETRY LATER" e il sistema attende 3 secondi prima di ritentare automaticamente la richiesta, tornando allo stato `simula_pressione_bottone`. Verifica quindi che il meccanismo di retry funzioni correttamente senza intervento manuale.

TEST 5 - Gestione della risposta "load refused"

```
State refused {
    onMsg( load_refused : loadRefused(none) ) {
        println("ioport | DISPLAY: load refused (hold is full)") color red
        [# displayMessage = "REFUSED - Hold Full" #]
    }
}
Goto display_idle
```

Copre il caso limite in cui la hold è completamente piena: il sistema risponde con `load_refused` e IOPort aggiorna il display segnalando il rifiuto. A differenza degli altri due rami, non c'è un nuovo tentativo automatico ma si torna direttamente allo stato `display_idle`, terminando il ciclo di test per questo scenario.

TEST 6 - Monitoraggio del trasferimento (in corso e completato)

```
State transfer_in_progress {
    println("ioport | DISPLAY: Transfer in progress") color cyan
    [# displayMessage = "Service working" #]
}
Transition t3
    whenMsg robot_complete_notification -> transfer_complete

State transfer_complete {
    println("ioport | DISPLAY: Transfer complete - Ready") color green
    [# displayMessage = "Transfer complete - Ready" #]
}
```

Verifica la fase finale del flusso di carico: dopo la `container_ack`, il display segnala che il trasferimento è in corso. Il test attende poi il `robot_complete` inviato da `cargoservice` al termine del trasferimento, e a quel punto aggiorna il display con "Transfer complete - Ready", chiudendo il ciclo di test completo del sistema.

6 Scelte Progettuali

L'obiettivo di questo sprint è quello di validare la logica di business e i flussi di coordinamento del sistema, muovendosi in modo indipendente dalla presenza dell'hardware fisico reale. Per fare questo in modo affidabile, l'architettura è stata strutturata seguendo il modello ad attori e distribuendo i componenti su due nodi

separati: **ctxcargo** e **ctxrobotsmart**. Questi nodi operano in totale isolamento di memoria e comunicano esclusivamente tramite messaggi, garantendo la robustezza e la scalabilità richieste per il prodotto finale.

Il fulcro di questa infrastruttura è il **cargoservice**, che opera come orchestratore centralizzato. Per testare a fondo la sua capacità di gestire l'intero flusso di lavoro senza dover attendere lo sviluppo del front-end, la porta di ingresso **IOPort** è stata progettata come un componente autonomo dotato di un piano di test integrato. Questo attore è in grado di simulare da solo, in sequenza temporale, la richiesta di carico, il posizionamento del container e il successivo rilevamento da parte del sensore, permettendo di verificare l'intero ciclo in modo automatico, deterministico e ripetibile.

Per quanto riguarda i dispositivi periferici, mentre il led e la gestione del magazzino nella **hold** sono già implementati con la loro logica interna completa, il sonar è configurato per monitorare ciclicamente le distanze e inviare un segnale di stop all'orchestratore solo in caso di anomalie prolungate.

Altri componenti non ancora cruciali in questo sprint, come il **markerdevice**, sono invece inseriti come scheletri logici e verranno sviluppati nelle fasi successive del progetto.

In questa versione, infatti, l'orchestratore farà fermare il robot davanti allo slot 5 per due secondi, simulandone un'etichettatura senza interagire davvero con il marker device.

Infine, l'architettura prevede una netta separazione tra la pianificazione del servizio e l'effettiva esecuzione dei movimenti del robot: l'attore locale **cargorobot** traduce i comandi dell'orchestratore in coordinate cartesiane concrete (X,Y). L'effettivo movimento sul campo viene poi delegato, tramite richieste mirate, all'attore esterno **robotsmart**, dislocato sul suo nodo indipendente e direttamente connesso. Tutta questa rete di comunicazione poggia su un broker MQTT operante sulla porta standard 1883.

6.1 Stato interno della hold

Per gestire la mappatura e la disponibilità degli spazi nel magazzino, l'attore **hold** mantiene al suo interno una rappresentazione esplicita e dinamica del proprio stato orientata agli oggetti. Questa gestione è affidata a una struttura dati di tipo **mutableMapOf**, che associa a ciascun identificativo testuale dello slot (da slot1 a slot4) il suo stato logico corrente.

Nel ciclo di vita del sistema, uno slot può assumere tre stati distinti:

- **free**: lo slot è vuoto e disponibile per essere prenotato. Tutti gli slot si trovano in questa condizione al momento dell'avvio del sistema.
- **reserved**: lo slot è stato opzionato dal **cargoservice** a seguito di una richiesta di verifica (**find_free_slot**). Questa transizione di stato intermedia garantisce che nessun'altra richiesta simultanea possa allocare lo stesso spazio mentre il container è ancora in viaggio verso l'area predisposta.
- **occupied**: il container è stato fisicamente depositato dal robot nella posizione corretta e il **cargoservice** ha confermato il completamento del trasferimento (**occupySlot**).

```
[#
    val slots = mutableMapOf(
        "slot1" to "free",
        "slot2" to "free",
        "slot3" to "free",
        "slot4" to "free"
    )
    var reservedSlot = ""
    var slotIsAvailable = false
#]
```

La stabilità di questa rappresentazione interna è garantita da una macchina a stati finiti che risponde alle richieste dell'orchestratore. Quando viene richiesto uno slot libero, la **hold** filtra la mappa per identificare il primo spazio in stato **free**, ne muta lo stato in **reserved** e ne restituisce l'ID.

Se il trasferimento va a buon fine, lo stato passa definitivamente a **occupied**; in caso di timeout o fallimento del sistema, l'attore è in grado di eseguire una procedura di rollback riportando lo slot specifico allo stato **free**.

6.2 Interazioni tra i componenti

Per quanto riguarda l'interazione tra **ioportmock** e **cargoservice**, sono stati già definiti nello Sprint 0 i messaggi utili all'interazione:

```
Request load_request : loadRequest(none)
Reply load_accepted : loadAccepted(slotID) for load_request
Reply load_retrylater : loadRetryLater(none) for load_request
Reply load_refused : loadRefused(none) for load_request
```

Per modellare l'evento fisico di rilevamento del container nell'IOPort dopo che il carico è stato accettato, cargoservice conferma la ricezione con una risposta che funge da via libera per far partire la richiesta di trasferimento verso il robot.

```
Request check_distance : checkDistance(none)
Reply distance_response : distanceResponse(DISTANCE) for check_distance
```

Per avviare il trasferimento, cargoservice invia una request a cargorobot per richiedere lo spostamento fisico del container verso lo slot riservato. La reply **robot_complete** chiude il ciclo di carico e permette a cargoservice di liberare lo stato "engaged" e notificare l'esito a IOPort.

```
Request robot_transfer : robotTransfer(SLOT)
Reply robot_complete : robotComplete(FINALSLOT) for robot_transfer
```

Cargoservice si occupa anche di definire l'intera API di gestione slot esposta dall'attore hold: ricerca di uno slot libero, occupazione del trasferimento del robot e rilascio in caso di timeout. Questi messaggi isolano completamente la logica di stato degli slot dagli altri attori, che la trattano come un servizio esterno.

```
Request find_free_slot : findFreeSlot(none)
Reply slot_found : slotFound(SLOTID) for find_free_slot
Reply slot_full : slotFull(none) for find_free_slot

Request find_occupy : occupySlot(SLOTID)
Reply occupy_done : occupySlotDone(none) for find_occupy

Request find_release : releaseSlot(SLOTID)
Reply release_done : releaseSlotDone(none) for find_release
```

Per l'interazione con gli altri dispositivi vengono usati dei dispatch per la gestione e l'aggiornamento dell'interfaccia utente: accensione e spegnimento del led, aggiornamento del display e notifica di fine trasferimento. Questo tipo di comunicazione non richiede conferma perché non altera la logica di stato del sistema, solo la sua rappresentazione esterna.

```
Dispatch sensor_data : sensorData(DISTANCE)
Dispatch led_blink : ledBlink(STATE)
Dispatch display_update : displayUpdate(MESSAGE)
Dispatch robot_complete_notification : robotCompleteNotif(FINALSLOT)
```

Per forzare una transizione interna in base a un risultato calcolato vengono definiti dei dispatch "auto-diretti" per implementare rami condizionali dentro lo stesso attore senza passare da uno stato esterno.

```
Dispatch slot_is_free : slot_is_free(none)
Dispatch slot_is_full : slot_is_full(none)
Dispatch sonar_normal : sonar_normal(none)
Dispatch sonar_warn : sonar_warn(none)
```


L'interazione con il componente esterno `robotSMART26` prevede comandi di movimento con relativo esito e comandi di calibrazione e posizionamento. Nel caso di trasferimento fisico non andato a buon fine, `cargorobot` risponde con una `robot_failed`.

```
Request moverobot : moverobot(TARGETX, TARGETY, STEPTIME)
Reply moverobotdone : moverobotdone(ARG) for moverobot
Reply robot_failed : robotFailed(ARG) for robot_transfer
Reply moverobotfailed : moverobotfailed(PLANDONE, PLANTODO) for moverobot

Request tuneAtHome : tuneAtHome(X)
Reply tuneDone : tuneDone(X) for tuneAtHome
Dispatch setrobotstate : setpos(X, Y, D)
```

6.3 Rappresentazione dell'attore `cargoservice`

L'attore `cargoservice` è inteso come una macchina a stati finiti che utilizza variabili interne per tracciare costantemente lo stato dei suoi componenti principali. A tale scopo, mantiene in memoria alcune informazioni fondamentali:

- **ioPortOccupied**: booleano che tiene traccia della disponibilità fisica della porta di ingresso dell'impianto. Viene impostata a **true** non appena viene riservato uno **slot** per un cliente, impedendo l'accettazione di richieste di carico simultanee, e ritorna **false** solo a trasferimento completato o in caso di fallimento.
- **CurrentSlot**: stringa che memorizza temporaneamente l'identificativo alfanumerico dello slot di destinazione assegnato dalla **hold**. Questa informazione è vitale per istruire correttamente il robot sulla coordinata finale di deposito.
- **ContainerDetected**: booleano che indica se il container è stato effettivamente posizionato sulla piattaforma di carico. Questa variabile viene valorizzata dinamicamente dal **cargoservice** analizzando i dati di distanza ricevuti a seguito dell'interrogazione attiva del **sonar**.

```
[#
    var ioPortOccupied = false
    var CurrentSlot = ""
    var ContainerDetected = false
#]
```

6.3.1 Gestione load-request del `cargoservice`

Quando il componente `cargoservice` riceve una richiesta di tipo `load_request`, esegue una verifica preliminare, tramite un costrutto condizionale esplicito in Kotlin/QAk, nello stato `handle_load_request` per valutare se la porta di ingresso è libera e se il sistema è operativo. Se queste condizioni non sono soddisfatte, il servizio transita verso lo stato `refuse_retry_later`, rispondendo al mittente con un messaggio di `load_retrylater` e tornando in attesa.

Se invece il controllo ha esito positivo, il componente passa allo stato `request_slot` e interroga la `hold` tramite la richiesta `find_free_slot`. In base alla risposta ricevuta nello stato di attesa `waiting_for_slot`: Se la `hold` è piena il servizio passa a `hold_is_full`, rifiuta la richiesta inviando un messaggio di `load_refused` e ritorna allo stato `disengaged`.

Se viene trovato uno slot libero il sistema passa allo stato `slot_available`, memorizza l'identificativo dello slot nella variabile **CurrentSlot**, imposta la porta di ingresso come occupata e invia la `load_accepted` al client.

Subito dopo, il sistema entra nello stato **engaged**, dove attiva il `led_blink` e inizia la comunicazione con il sonar.

```

State handle_load_request {
    println("ioport -> cargoservice | load request") color cyan
}
Goto request_slot if [# !ioPortOccupied #] else refuse_retry_later

State request_slot {
    request hold -m find_free_slot : findFreeSlot(none)
}
Goto waiting_for_slot

State refuse_retry_later {
    replyTo load_request with load_retrylater : loadRetryLater(none)
}
Goto disengaged

State waiting_for_slot {
    println("cargoservice | waiting for slot response") color cyan
}
Transition t1
    whenReply slot_found -> slot_available
    whenReply slot_full -> hold_is_full

State slot_available {
    onMsg( slot_found : slotFound(SLOTID) ) {
        [#
            CurrentSlot = payloadArg(0).toString()
            ioPortOccupied = true
        #]
        println("cargoservice | reserved $CurrentSlot") color green
        replyTo load_request with load_accepted : loadAccepted($CurrentSlot)
    }
}
Goto engaged

State hold_is_full {
    println("cargoservice | hold is full") color red
    replyTo load_request with load_refused : loadRefused(none)
}
Goto disengaged

```

6.3.2 Gestione degli eventi del sonar

Coerentemente con quanto stabilito in fase di analisi, il cargoservice non interroga attivamente il sonar per conoscere la distanza misurata, ma si limita a reagire alle notifiche che quest'ultimo gli invia in modo autonomo, eliminando ogni forma di attesa ciclica lato orchestratore.

Non appena una richiesta di carico viene accettata e lo slot in stiva viene formalmente riservato, il cargoservice attiva l'indicatore visivo del **led** e transita nello stato **engaged**, dove rimane in attesa passiva di un'unica notifica: la Request **container_detected**. Tale messaggio viene inoltrato al cargoservice solo nel momento in cui la presenza del container è effettivamente accertata, e la relativa Reply **container_ack** chiude il ciclo di conferma, portando il sistema nello stato **container_arrived** da cui parte la richiesta di trasferimento al robot.

Parallelamente, il sonar esegue un monitoraggio interno indipendente, basato su un ciclo temporizzato (**whenTime**) che confronta periodicamente la distanza rilevata con la costante di soglia **DFREE** e tiene traccia del numero di rilevazioni anomale consecutive. Solo qualora tale condizione di guasto persista oltre la soglia prevista, il sonar inoltra al cargoservice il Dispatch **sensor_data**, che porta il sistema nello stato **update_status** e ne segnala l'uscita fuori servizio. In assenza di anomalie, il sonar non genera alcun traffico verso il cargoservice: il monitoraggio della distanza resta interamente incapsulato nella logica interna del sonar, che notifica l'esterno esclusivamente in presenza di eventi rilevanti ai fini dell'orchestrazione.

DA CAMBIARE IL CODICE

```
State engaged {
  println("cargoservice | waiting for container - Interrogo il sonar...")
  color yellow
  forward led -m led_blink : ledBlink(on)
  // Chiede la distanza al sonar
  request sonar -m check_distance : checkDistance(none)
}
Transition t2
  whenReply distance_response -> evaluate_distance

State evaluate_distance {
  onMsg( distance_response : distanceResponse(DISTANCE) ) {
    [#
      val Dist = payloadArg(0).toString().toInt()
      ContainerDetected = Dist < 80
    #]
  }
}
Goto container_arrived if [# ContainerDetected #] else retry_check

State retry_check {
  println("cargoservice | Container non ancora rilevato, riprovo
  tra 2 secondi...") color yellow
  delay 2000
}
Goto engaged
```

6.3.3 Gestione dei movimenti del robot

Il coordinamento delle attività di movimentazione del container è guidato dal cargoservice attraverso un protocollo di richiesta-risposta con l'attore **cargorobot**. Una volta che il container viene rilevato nella zona di ingresso, il servizio invia una request di **robot_transfer** al cargorobot, inoltrando come argomento l'identificativo dello slot precedentemente riservato. Il cargoservice rimane quindi in attesa nello stato **container_arrived** finché il robot non completa il trasferimento fisico del carico dall'area di input alla

destinazione. La ricezione della risposta `robot_complete` sblocca il flusso, portando il sistema nello stato `on_robot_completed`. A questo punto, viene spento il led di segnalazione e viene inviata una richiesta alla hold per registrare formalmente l'occupazione dello slot. Solo dopo aver ricevuto la conferma di avvenuta registrazione, il sistema notifica l'IOPort del completamento dell'operazione tramite il messaggio `robotCompleteNotif` e reimposta le variabili di stato per prepararsi a una nuova sessione.

```

State container_arrived {
    // Non serve la replyTo container_detected poiché il trigger è interno (sonar)
    println("cargoservice | Container detected via sonar -
    requesting robot transfer") color green
    request cargorobot -m robot_transfer : robotTransfer($CurrentSlot)
}
Transition t3
    whenReply robot_complete -> on_robot_completed

State on_robot_completed {
    forward led -m led_blink : ledBlink(off)
    request hold -m find_occupy : occupySlot($CurrentSlot)
}
Transition t4
    whenReply occupy_done -> finalize_load_ok

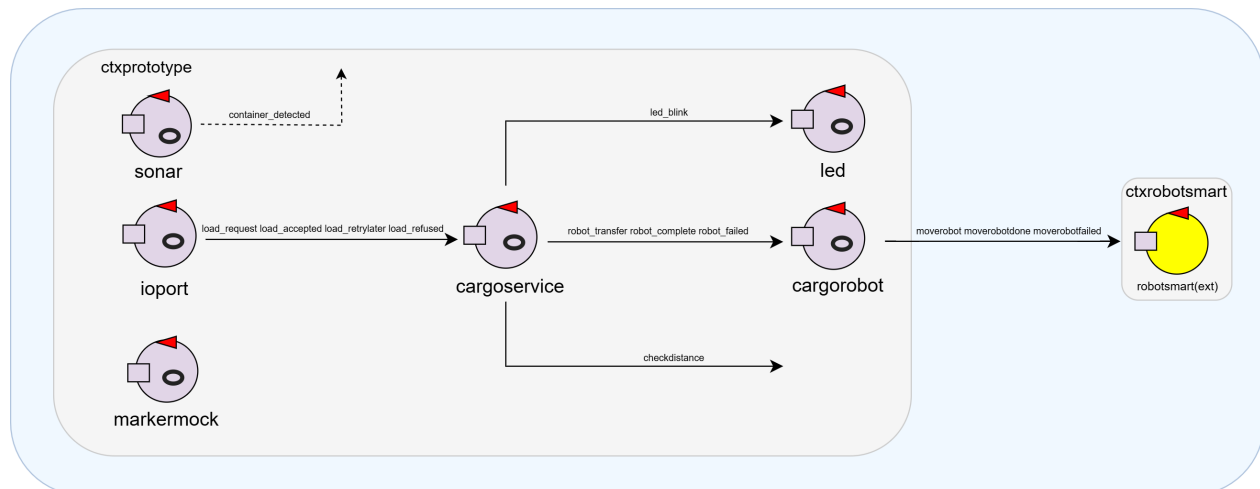
State finalize_load_ok {
    [#
        ioPortOccupied = false
        var SlotSend = CurrentSlot
        CurrentSlot = ""
    #]
    println("cargoservice | transfer completed and slot occupied") color green
    forward ioport -m robot_complete_notification : robotCompleteNotif($SlotSend)
}
Goto disengaged

```

7 Sintesi

7.1 Architettura del Sistema

L'immagine seguente riporta l'architettura finale del prototipo.



7.2 Istruzioni sul Deployment

Per procedere al deployment del prototipo è necessario effettuare i passaggi qui riportati:

1. Clonare il repository del progetto da GitHub
2. Nella directory `robotsmart26/yamls` eseguire il comando `docker-compose -f unibobasic26.yaml up`
3. Aprire un browser e andare su `http://localhost:8090/`
4. Eseguire all'interno della directory del progetto `robotsmart26` il comando `./gradlew run`
5. Eseguire all'interno della directory del progetto `sprint1` il comando `./gradlew run`

7.3 Verifica esecuzione della pianificazione

La pianificazione prevista è stata rispettata.

7.4 Goal dello Sprint 2

L'obiettivo dello Sprint 2 sarà quello di sostituire i componenti mock del prototipo con implementazioni più adatte e realistiche. In particolare:

- Gestire e integrare i componenti sonar e led al Pico W fisico;
- Realizzare la porta IOPort come Web-GUI.
- Integrare l'attesa di 30 secondi tra l'accettazione della richiesta e il rilevamento del container sull'IOPort.
- Implementare il comportamento del marker device.